

Kelompok Cemut Tugas 2

Anggota Kelompok :

Septya Noer Fauziyah	103012400084
Zalika Zahra Tasyifa Poerba	103012400311
Jihan Putri Ferdina	103012400355

1. Soal Segitiga

```
#include <stdio.h>
#include <math.h>
#include <stdbool.h>

static void urutkan(double a, double b, double c, double *big, double *s1, double *s2) {
    if (a >= b && a >= c) { *big = a; *s1 = b; *s2 = c; }
    else if (b >= a && b >= c) { *big = b; *s1 = a; *s2 = c; }
    else { *big = c; *s1 = a; *s2 = b; }
}

void segitigaBulat(long long a, long long b, long long c) {
    long long big, s1, s2;
    if (a >= b && a >= c) { big = a; s1 = b; s2 = c; }
    else if (b >= a && b >= c) { big = b; s1 = a; s2 = c; }
    else { big = c; s1 = a; s2 = b; }

    if (a <= 0 || b <= 0 || c <= 0) {
        printf("Tidak bisa dibangun segitiga (ada sisi <= 0)\n");
        return;
    }
    if (big >= s1 + s2) {
        printf("Tidak bisa dibangun segitiga (sisi terbesar >= jumlah dua sisi lain)\n");
        return;
    }

    bool sisiSama = (a == b && b == c);
    bool samaKaki = (a == b || b == c || a == c) && !sisiSama;
    bool siku = (big * big == s1 * s1 + s2 * s2);

    if (sisiSama) printf("SAMA SISI (EQUILATERAL)\n");
    if (samaKaki) printf("SAMA KAKI (ISOSCELES)\n");
    if (siku) printf("SIKU-SIKU (RIGHT TRIANGLE)\n");
    if (!sisiSama && !samaKaki && !siku)
        printf("SEMBARANG (FREE TRIANGLE)\n");
}

#define AKURASI 0.01

static bool sama(double x, double y) {
    double m = (x > y) ? x : y;
    return fabs(x - y) <= AKURASI * m;
}
```

```

void segitigaPecahan(double a, double b, double c) {
    double big, s1, s2;
    urutkan(a, b, c, &big, &s1, &s2);

    if (a <= 0 || b <= 0 || c <= 0) {
        printf("Tidak bisa dibangun segitiga (ada sisi <= 0)\n");
        return;
    }
    if (big >= s1 + s2) {
        printf("Tidak bisa dibangun segitiga (sisi terbesar >= jumlah dua sisi lain)\n");
        return;
    }

    bool ab = sama(a, b), bc = sama(b, c), ac = sama(a, c);
    bool sisiSama = (ab && bc);
    bool samaKaki = (ab || bc || ac) && !sisiSama;
    /* siku-siku: sisi terbesar ~ sqrt(s1^2 + s2^2) dalam toleransi 1% */
    bool siku = sama(big, sqrt(s1 * s1 + s2 * s2));

    if (sisiSama) printf("SAMA SISI (EQUILATERAL)\n");
    if (samaKaki) printf("SAMA KAKI (ISOSCELES)\n");
    if (siku)      printf("SIKU-SIKU (RIGHT TRIANGLE)\n");
    if (!sisiSama && !samaKaki && !siku)
        printf("SEMBARANG (FREE TRIANGLE)\n");
}

int main(void) {
    int pilih;
    printf("1. Input bilangan bulat\n2. Input bilangan pecahan (akurasi 1%)\nPilih: ");
    if (scanf("%d", &pilih) != 1) return 1;

    if (pilih == 1) {
        long long a, b, c;
        printf("Masukkan a b c: ");
        if (scanf("%lld %lld %lld", &a, &b, &c) != 3) return 1;
        segitigaBulat(a, b, c);
    } else {
        double a, b, c;
        printf("Masukkan a b c: ");
        if (scanf("%lf %lf %lf", &a, &b, &c) != 3) return 1;
        segitigaPecahan(a, b, c);
    }
    return 0;
}

```

SEGITIGA SAMA SISI

```

1. Input bilangan bulat
2. Input bilangan pecahan (akurasi 1%)
Pilih: 1
Masukkan a b c: 5 5 5
SAMA SISI (EQUILATERAL)

Process returned 0 (0x0)   execution time : 4.841 s
Press any key to continue.

```

SEGITIGA SIKU-SIKU

```

1. Input bilangan bulat
2. Input bilangan pecahan (akurasi 1%)
Pilih: 1
Masukkan a b c: 3 4 5
SIKU-SIKU (RIGHT TRIANGLE)

Process returned 0 (0x0)   execution time : 22.021 s
Press any key to continue.

```

Segitiga Sama Kaki & Siku-Siku Sekaligus

```
1. Input bilangan bulat
2. Input bilangan pecahan (akurasi 1%)
Pilih: 2
Masukkan a b c: 10 10 14.14
SAMA KAKI (ISOSCELES)
SIKU-SIKU (RIGHT TRIANGLE)

Process returned 0 (0x0)    execution time : 19.853 s
Press any key to continue.
```

Sembarang

```
1. Input bilangan bulat
2. Input bilangan pecahan (akurasi 1%)
Pilih: 1
Masukkan a b c: 4 6 9
SEMBARANG (FREE TRIANGLE)

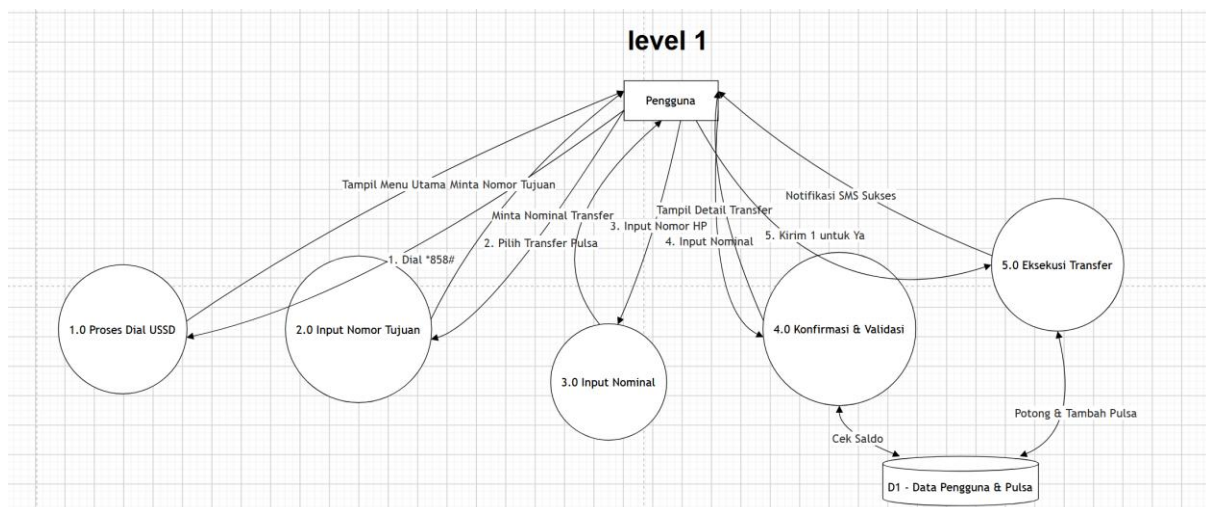
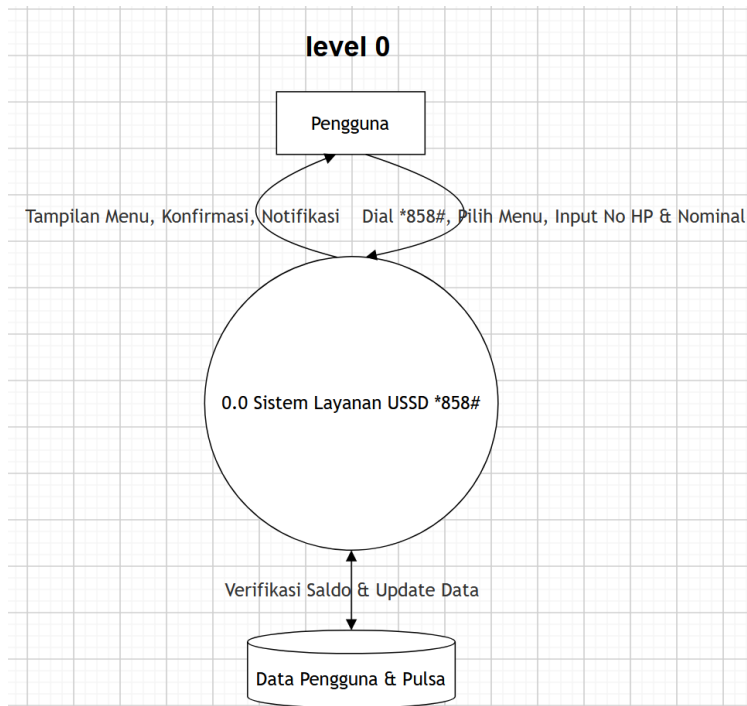
Process returned 0 (0x0)    execution time : 4.408 s
Press any key to continue.
```

Bukan Segitiga

```
1. Input bilangan bulat
2. Input bilangan pecahan (akurasi 1%)
Pilih: 1
Masukkan a b c: 1 2 5
Tidak bisa dibangun segitiga (sisi terbesar >= jumlah dua sisi lain)

Process returned 0 (0x0)    execution time : 3.827 s
Press any key to continue.
```

2. A. Untuk layanan *858#, pelajari pemakaiannya, perhatikan IMKnya, level menu-menu berikutnya, sampai eksekusi



B. PSPEC (Structured English)

Jalur layanan yang dipilih: *858# → 1.Transfer Pulsa → input nomor tujuan → input nominal → konfirmasi → eksekusi → notifikasi. Proses 2.0 pada Level 1 dipecah menjadi lima proses primitif (2.1–2.5) di Level 2, dan masing-masing memiliki PSPEC. Nilai aturan bisnis (nominal minimum, biaya admin, dll.) adalah asumsi simulasi.

PSPEC 2.1 – Validasi Nomor Tujuan

Nomor / Nama	2.1 Validasi Nomor Tujuan
Input	Nomor tujuan (teks) dari Pengirim; status nomor dari D1
Output	Nomor tujuan ternormalisasi (08xxxx) atau pesan error

```
PROCESS Validasi Nomor Tujuan
  Normalisasi nomor: hapus spasi, '-' dan '+'
  IF nomor diawali '62' THEN ganti awalan '62' dengan '0'
  IF nomor bukan angka atau tidak sesuai pola 08xxxxxxx (10-13 digit) THEN
    tampilkan 'Format nomor tidak valid' dan akhiri sesi
  ELSE IF nomor tujuan = nomor pengirim THEN
    tampilkan 'Tidak dapat transfer ke nomor sendiri' dan akhiri sesi
  ELSE IF nomor tujuan tidak ada di D1 OR status nomor tidak aktif THEN
    tampilkan 'Nomor tujuan tidak terdaftar / tidak aktif' dan akhiri sesi
  ELSE
    teruskan nomor tujuan ke proses 2.2
  ENDIF
END PROCESS
```

PSPEC 2.2 – Validasi Nominal & Saldo

Nomor / Nama	2.2 Validasi Nominal & Saldo
Input	Nominal (teks) dari Pengirim; saldo pengirim dari D1
Output	Nominal valid atau pesan error

```

PROCESS Validasi_Nominal_dan_Saldo
  Hapus tanda '.' pada nominal
  IF nominal bukan angka THEN tampilkan 'Nominal harus berupa angka'
  ELSE IF nominal < MIN_TRANSFER OR nominal > MAKS_TRANSFER THEN
    tampilkan 'Nominal di luar batas'
  ELSE IF nominal bukan kelipatan KELIPATAN THEN
    tampilkan 'Nominal harus kelipatan Rp1.000'
  ELSE IF saldo pengirim - (nominal + BIAYA_ADMIN) < MIN_SISA_SALDO THEN
    tampilkan 'Pulsa tidak mencukupi'
  ELSE
    teruskan nominal ke proses 2.3
  ENDIF
  (jika ada error: sesi diakhiri tanpa mengubah data)
END PROCESS

```

PSPEC 2.3 – Konfirmasi Transaksi

Nomor / Nama	2.3 Konfirmasi Transaksi
Input	Nomor tujuan valid, nominal valid, jawaban Pengirim
Output	Transaksi disetujui atau dibatalkan

```

PROCESS Konfirmasi_Transaksi
  total = nominal + BIAYA_ADMIN
  tampilkan 'Transfer Pulsa <nominal> ke <nomor>. Total <total>. 1.Ya 2.Tidak'
  baca jawaban Pengirim
  IF jawaban = 1 THEN
    teruskan transaksi ke proses 2.4
  ELSE
    tampilkan 'Transfer pulsa dibatalkan' dan akhiri sesi
  ENDIF
END PROCESS

```

PSPEC 2.4 – Eksekusi Transfer

Nomor / Nama	2.4 Eksekusi Transfer
Input	Transaksi disetujui; saldo pengirim dan penerima dari D1
Output	Status transaksi (SUKSES/GAGAL); update D1, D2, D4

```

PROCESS Eksekusi_Transfer
  total = nominal + BIAYA_ADMIN
  IF saldo pengirim - total < MIN_SISA_SALDO THEN      (cek ulang)
    status = GAGAL
  ELSE
    saldo pengirim = saldo pengirim - total
    saldo penerima = saldo penerima + nominal
    tambah kupon undian untuk pengirim di D4
    status = SUKSES
  ENDIF
  simpan (waktu, pengirim, penerima, nominal, biaya, status) ke D2
  teruskan status ke proses 2.5
END PROCESS

```

PSPEC 2.5 – Kirim Notifikasi

Nomor / Nama	2.5 Kirim Notifikasi
Input	Status transaksi, saldo terbaru
Output	Notifikasi ke Pengirim; SMS ke Penerima

```

PROCESS Kirim_Notifikasi
  IF status = SUKSES THEN
    tampilkan ke Pengirim 'Transfer pulsa <nominal> ke <nomor> BERHASIL. Sisa pulsa <saldo>'
    kirim SMS ke Penerima 'Anda menerima pulsa <nominal> dari <pengirim>'
  ELSE
    tampilkan ke Pengirim 'Transfer pulsa gagal. Silakan coba lagi'
  ENDIF
END PROCESS

```

C. Implementasi Code Dari yang B

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <ctype.h>
#include <time.h>

// =====
// KONFIGURASI (asumsi simulasi)
// =====

#define KODE USSD "*858#"
#define MIN_TRANSFER 5000 // nominal minimum (Rp)
#define MAKS_TRANSFER 100000 // nominal maksimum per transaksi (Rp)
#define KELIPATAN 1000 // nominal harus kelipatan ini
#define BIAYA_ADMIN 1500 // biaya admin dibebankan ke pengirim (Rp)
#define MIN_SISA_SALDO 0 // sisa saldo minimum setelah transfer (Rp)

#define MAX_PELANGGAN 50
#define MAX_TRANSAKSI 100
#define LENGTH_NOMOR 20
#define LENGTH_WAKTU 30

const char *MENU_UTAMA =
    "Layanan *858#\n"
    "1.Transfer Pulsa\n"
    "2.Minta Pulsa\n"
    "3.Auto TP\n"
    "4.Delete Auto TP\n"
    "5.List Auto TP\n"
    "6.Cek Kupon Undian TP";

const char *PROMPT_NOMOR = "Silahkan masukkan nomor tujuan Transfer Pulsa : (contoh: 08xxxx atau 628xxxx)";

// =====
// DATA STRUCTURES (pengganti database)
// =====

typedef struct {
    char nomor[LENGTH_NOMOR];
    int saldo;
    bool aktif;
} Pelanggan;

typedef struct {
    char waktu[LENGTH_WAKTU];
    char pengirim[LENGTH_NOMOR];
    char penerima[LENGTH_NOMOR];
    int nominal;
    int biaya;
    char status[10]; // "SUKSES" atau "GAGAL"
} Transaksi;

typedef struct {
    Pelanggan pelanggan[MAX_PELANGGAN];
    int jumlah_pelanggan;
    Transaksi transaksi[MAX_TRANSAKSI];
    int jumlah_transaksi;
} Database;

// Helper: Tambah pelanggan ke Database
void tambah_pelanggan(Database *db, const char *nomor, int saldo, bool aktif) {
    if (db->jumlah_pelanggan < MAX_PELANGGAN) {
        strcpy(db->pelanggan[db->jumlah_pelanggan].nomor, nomor);
        db->pelanggan[db->jumlah_pelanggan].saldo = saldo;
        db->pelanggan[db->jumlah_pelanggan].aktif = aktif;
        db->jumlah_pelanggan++;
    }
}
```

```

// Helper: Cari pelanggan berdasarkan nomor
Pelanggan* cari_pelanggan(Database *db, const char *nomor) {
    for (int i = 0; i < db->jumlah_pelanggan; i++) {
        if (strcmp(db->pelanggan[i].nomor, nomor) == 0) {
            return &db->pelanggan[i];
        }
    }
    return NULL;
}

// =====
// PROSES 2.1: Validasi Nomor Tujuan
// =====

bool normalisasi_nomor(const char *teks, char *output) {
    char cleaned[100] = "";
    int idx = 0;

    // Bersihkan spasi dan strip
    for (int i = 0; teks[i] != '\0'; i++) {
        if (teks[i] != ' ' && teks[i] != '-' && teks[i] != '\n' && teks[i] != '\r') {
            cleaned[idx++] = teks[i];
        }
    }
    cleaned[idx] = '\0';

    char *p = cleaned;
    if (p[0] == '+') p++;

    // Cek apakah semua karakter berupa angka
    for (int i = 0; p[i] != '\0'; i++) {
        if (!isdigit(p[i])) return false;
    }

    // Ubah awalan 62 menjadi 0
    if (strncmp(p, "62", 2) == 0) {
        output[0] = '0';
        strcpy(output + 1, p + 2);
    } else {
        strcpy(output, p);
    }

    // Validasi panjang dan awalan 08 (total 10-13 digit)
    size_t len = strlen(output);
    if (strncmp(output, "08", 2) == 0 && len >= 10 && len <= 13) {
        return true;
    }

    return false;
}

bool validasi_nomor_tujuan(Database *db, const char *teks, const char *nomor_pengirim, char *nomor_out, char *pesan_error) {
    if (!normalisasi_nomor(teks, nomor_out)) {
        strcpy(pesan_error, "Format nomor tidak valid. Gunakan 08xxxx atau 628xxxx.");
        return false;
    }

    if (strcmp(nomor_out, nomor_pengirim) == 0) {
        strcpy(pesan_error, "Tidak dapat transfer pulsa ke nomor sendiri.");
        return false;
    }

    Pelanggan *penerima = cari_pelanggan(db, nomor_out);
    if (penerima == NULL || !penerima->aktif) {
        strcpy(pesan_error, "Nomor tujuan tidak terdaftar / tidak aktif.");
        return false;
    }
}

```

```

    return true;
}

// =====
// PROSES 2.2: Validasi Nominal dan Saldo
// =====
bool validasi_nominal(Database *db, const char *teks, const char *nomor_pengirim, int *nominal_out, char *pesan_error) {
    char cleaned[50] = "";
    int idx = 0;

    // Bersihkan titik, spasi, dan newline
    for (int i = 0; teks[i] != '\0'; i++) {
        if (teks[i] != '.' && teks[i] != ' ' && teks[i] != '\n' && teks[i] != '\r') {
            if (!isdigit(teks[i])) {
                strcpy(pesan_error, "Nominal harus berupa angka.");
                return false;
            }
            cleaned[idx++] = teks[i];
        }
    }
    cleaned[idx] = '\0';

    if (strlen(cleaned) == 0) {
        strcpy(pesan_error, "Nominal harus berupa angka.");
        return false;
    }

    int nominal = atoi(cleaned);

    if (nominal < MIN_TRANSFER || nominal > MAKS_TRANSFER) {
        sprintf(pesan_error, "Nominal harus antara Rp%d dan Rp%d.", MIN_TRANSFER, MAKS_TRANSFER);
        return false;
    }

    if (nominal % KELIPATAN != 0) {
        sprintf(pesan_error, "Nominal harus kelipatan Rp%d.", KELIPATAN);
        return false;
    }

    Pelanggan *pengirim = cari_pelanggan(db, nomor_pengirim);
    if (pengirim == NULL || (pengirim->saldo - (nominal + BIAAYA_ADMIN) < MIN_SISA_SALDO)) {
        strcpy(pesan_error, "Pulsa Anda tidak mencukupi untuk transaksi ini.");
        return false;
    }

    *nominal_out = nominal;
    return true;
}

// =====
// PROSES 2.3: Konfirmasi Transaksi
// =====
void cetak_konfirmasi(const char *nomor_tujuan, int nominal) {
    printf("Transfer Pulsa Rp%d ke %s.\n", nominal, nomor_tujuan);
    printf("Biaya admin Rp%d. Total Rp%d.\n", BIAAYA_ADMIN, nominal + BIAAYA_ADMIN);
    printf("1.Ya\n2.Tidak\n");
}

// =====
// PROSES 2.4: Eksekusi Transfer
// =====
Transaksi eksekusi_transfer(Database *db, const char *pengirim, const char *penerima, int nominal) {
    int total = nominal + BIAAYA_ADMIN;
    Pelanggan *p_kirim = cari_pelanggan(db, pengirim);
    Pelanggan *p_terima = cari_pelanggan(db, penerima);

```

```

Transaksi trx;

// Dapatkan waktu saat ini
time_t now = time(NULL);
struct tm *t = localtime(&now);
strftime(trx.waktu, sizeof(trx.waktu), "%Y-%m-%d %H:%M:%S", t);

strcpy(trx.pengirim, pengirim);
strcpy(trx.penerima, penerima);
trx.nominal = nominal;
trx.biaya = BIAYA_ADMIN;

// Cek ulang saldo sebelum eksekusi
if (p_kirim == NULL || (p_kirim->saldo - total < MIN_SISA_SALDO)) {
    strcpy(trx.status, "GAGAL");
} else {
    p_kirim->saldo -= total;
    if (p_terima != NULL) {
        p_terima->saldo += nominal;
    }
    strcpy(trx.status, "SUKSES");
}

if (db->jumlah_transaksi < MAX_TRANSAKSI) {
    db->transaksi[db->jumlah_transaksi++] = trx;
}

return trx;
}

// =====
// PROSES 2.5: Kirim Notifikasi
// =====
void notifikasi(Database *db, Transaksi trx) {
    if (strcmp(trx.status, "SUKSES") != 0) {
        printf("Transfer pulsa gagal. Silakan coba lagi.\n");
        return;
    }

    Pelanggan *p_kirim = cari_pelanggan(db, trx.pengirim);
    int sisa = (p_kirim != NULL) ? p_kirim->saldo : 0;

    printf("Transfer pulsa Rp%d ke %s BERHASIL. Sisa pulsa Rp%d.\n", trx.nominal, trx.penerima, sisa);
    printf("[SMS ke %s] Anda menerima pulsa Rp%d dari %s.\n", trx.penerima, trx.nominal, trx.pengirim);
}

// =====
// PROSES 1.0: Sesi USSD (menu -> pilihan -> sampai eksekusi)
// =====
void alur_transfer_pulsa(Database *db, const char *pengirim) {
    char input_buf[100];
    char nomor_tujuan[LENGTH_NOMOR];
    char pesan_error[150];
    int nominal = 0;

    // Level 2: Nomor Tujuan
    printf("%s\n> ", PROMPT_NOMOR);
    if (!fgets(input_buf, sizeof(input_buf), stdin)) return;

    if (!validasi_nomor_tujuan(db, input_buf, pengirim, nomor_tujuan, pesan_error)) {
        printf("%s\n", pesan_error);
        return;
    }
}

```

```

// Level 3: Nominal
printf("Masukkan nominal Transfer Pulsa (Rp%d - Rp%d, kelipatan Rp%d):\n> ", MIN_TRANSFER, MAKS_TRANSFER, KELIPATAN);
if (!fgets(input_buf, sizeof(input_buf), stdin)) return;

if (!validasi_nominal(db, input_buf, pengirim, &nominal, pesan_error)) {
    printf("%s\n", pesan_error);
    return;
}

// Level 4: Konfirmasi
cetak_konfirmasi(nomor_tujuan, nominal);
printf("> ");
if (!fgets(input_buf, sizeof(input_buf), stdin)) return;

// Hapus whitespace/newline dari input konfirmasi
char pil = input_buf[0];
if (pil != '1') {
    printf("Transfer pulsa dibatalkan.\n");
    return;
}

// Eksekusi + Notifikasi
Transaksi trx = eksekusi_transfer(db, pengirim, nomor_tujuan, nominal);
notifikasi(db, trx);
}

void sesi_usdd(Database *db, const char *pengirim) {
    char input_buf[50];

    printf("Dial %s\n\n", KODE_USSD);
    printf("%s\n> ", MENU_UTAMA);

    if (!fgets(input_buf, sizeof(input_buf), stdin)) return;

    // Ambil opsi pilihan pertama
    int pilihan = atoi(input_buf);

    if (pilihan == 1) {
        alur_transfer_pulsa(db, pengirim);
    } else if (pilihan >= 2 && pilihan <= 6) {
        printf("Menu ini belum diimplementasikan pada simulasi.\n");
    } else {
        printf("Pilihan tidak valid.\n");
    }
}

// =====
// MAIN PROGRAM
// =====

int main(void) {
    Database db = { .jumlah_pelanggan = 0, .jumlah_transaksi = 0 };

    // Inisialisasi Data Pelanggan (Simulasi Database)
    tambah_pelanggan(&db, "081234567890", 50000, true); // Pengirim
    tambah_pelanggan(&db, "082111222333", 3000, true); // Penerima aktif
    tambah_pelanggan(&db, "085777888999", 10000, false); // Penerima tidak aktif

    printf("=== SIMULASI USSD *858# (pengirim: 081234567890, saldo Rp50.000) ===\n");
    printf("Nomor tujuan uji: 082111222333 (aktif), 085777888999 (tidak aktif)\n\n");

    sesi_usdd(&db, "081234567890");

    Pelanggan *p_kirim = cari_pelanggan(&db, "081234567890");
    Pelanggan *p_terima = cari_pelanggan(&db, "082111222333");

    Pelanggan *p_kirim = cari_pelanggan(&db, "081234567890");
    Pelanggan *p_terima = cari_pelanggan(&db, "082111222333");

    printf("\nSaldo akhir pengirim : Rp%d\n", p_kirim ? p_kirim->saldo : 0);
    printf("Saldo akhir penerima : Rp%d\n", p_terima ? p_terima->saldo : 0);

    return 0;
}

```

```
=== SIMULASI USSD *858# (pengirim: 081234567890, saldo Rp50,000) ===  
Nomor tujuan uji: 082111222333 (aktif), 085777888999 (tidak aktif)  
  
Dial *858#  
  
Layanan *858#  
1.Transfer Pulsa  
2.Minta Pulsa  
3.Auto TP  
4.Delete Auto TP  
5.List Auto TP  
6.Cek Kupon Undian TP  
> 1  
Silahkan masukkan nomor tujuan Transfer Pulsa : (contoh: 08xxxx atau 628xxxx)  
> 6282111222333  
Masukkan nominal Transfer Pulsa (Rp5000 - Rp100000, kelipatan Rp1000):  
> 20.000  
Transfer Pulsa Rp20000 ke 082111222333.  
Biaya admin Rp1500. Total Rp21500.  
1.Ya  
2.Tidak  
> 1  
Transfer pulsa Rp20000 ke 082111222333 BERHASIL. Sisa pulsa Rp28500.  
[SMS ke 082111222333] Anda menerima pulsa Rp20000 dari 081234567890.  
  
Saldo akhir pengirim : Rp28500  
Saldo akhir penerima : Rp23000  
  
Process returned 0 (0x0)   execution time : 33.168 s  
Press any key to continue.  
|
```